

An Automated ETL Pipeline for Healthcare Analytics Using Pentaho Data Integration and Power BI

Edbert Dario
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta 11480
Indonesia
edbert.dario@binus.ac.id

Aldreine Vanda Kauntu
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta 11480
Indonesia
aldreine.kauntu@binus.ac.id

Abstract—This paper presents the design and implementation of an end-to-end data engineering pipeline applied to a synthetic healthcare dataset comprising 55,500 patient records. The pipeline covers all nine stages of the data engineering lifecycle: data source identification, ingestion, storage, transformation, data modeling, data quality and validation, automation and orchestration, data consumption, and evaluation. Raw data is extracted from a CSV file using Pentaho Data Integration (PDI), cleansed and validated through a sequence of native Pentaho transformation steps, stored in a two-layer MySQL architecture (staging and warehouse schemas), modeled using a star schema with one fact table and seven dimension tables, and served through a Power BI dashboard. Automation is handled by a Pentaho job that enforces dependency checks and error logging. Findings demonstrate that a structured, step-driven ETL pipeline using Pentaho improves data reliability, maintainability, and analytical readiness compared to SQL-centric transformation approaches.

Keywords—data engineering, ETL pipeline, Pentaho Data Integration, Mysql, star schema, Power BI, healthcare analytics, data quality.

I. INTRODUCTION

Healthcare organizations today generate enormous amounts of data from patient records, clinical encounters, billing systems, and diagnostic platforms. Despite this, a well-recognized gap exists between how much data is collected and how effectively it is used to support decisions [1]. The core issue is that raw healthcare data is scattered across different systems, often inconsistent, and rarely structured in a way that allows for straightforward analysis. Bridging this gap requires more than just storing data; it requires a systematic process for organizing and preparing it so that it can produce reliable and meaningful insights [2].

A widely adopted solution to this problem is the data warehouse, which consolidates data from various sources into a single, structured repository designed for analytical queries. Star schema is one of the most common approaches used in healthcare data warehousing, organizing data into a central fact table surrounded by dimension tables that provide descriptive context. Martins and Rangel [3] noted that data warehouses

enable healthcare researchers and managers to run complex analyses that would be difficult or slow on raw operational databases. When properly implemented, this kind of structure allows organizations to ask questions across multiple dimensions of their data, such as comparing patient outcomes by condition, hospital, or time, without sacrificing query performance.

Before data can be loaded into a warehouse, it must go through a transformation process that addresses quality issues such as duplicates, missing values, and inconsistencies. This process is commonly known as Extract, Transform, Load, or ETL, and its quality directly determines the reliability of downstream analytics [4]. Once the data is clean and stored, it still needs to be presented in a way that is accessible to stakeholders who are not technical experts. Interactive dashboards built with tools such as Power BI address this need by visualizing key metrics in a format that can be explored and filtered in real time [5]. However, most existing studies treat these components separately rather than as parts of a single integrated system, leaving a gap in practical, end-to-end implementations that can be reproduced and adapted.

This study addresses that gap by designing and implementing a complete healthcare analytics system built on a synthetic patient encounter dataset from Kaggle. The system covers the full pipeline from raw data to interactive dashboard, using Pentaho Data Integration for ETL processing, MySQL for structured storage based on star schema, and Microsoft Power BI for visualization. The goal is to produce a reproducible, integrated implementation that demonstrates how these three components can work together as a coherent analytics platform.

II. RELATED WORK

A. Healthcare Data Management

Healthcare organizations generate large volumes of clinical data from multiple sources such as electronic health records, billing systems, and admission databases, each with different formats and structures. This heterogeneity makes it difficult to consolidate data into a form that is suitable for analysis [2]. Understanding these challenges justifies the need for a

structured pipeline that organizes raw patient encounter data into a unified and analytically ready repository.

B. ETL Pipeline

ETL is the process of extracting data from a source, transforming it through cleaning and validation, and loading it into a target system. Seenivasan [6] noted that effective ETL pipelines must enforce data quality checks at the transformation stage to ensure completeness, consistency, and accuracy before data reaches its destination. In this study, ETL is implemented using Pentaho Data Integration to automate this process across repeated pipeline executions.

C. Data Quality in Healthcare

Data quality in healthcare covers dimensions such as completeness, accuracy, consistency, and currency. Syed et al. [7] found through a systematic review that incomplete and inconsistent records are the most pervasive problems in digital health data, directly undermining the reliability of any downstream analysis. This study addresses these issues within the ETL pipeline through a multi-stage validation process that removes duplicates, corrects inconsistencies, and filters out logically invalid records.

D. Data Warehouse and Star Schema

A data warehouse is a repository designed for analytical querying rather than transactional processing. The star schema is a widely used dimensional modeling approach that places a fact table at the center, surrounded by dimension tables that provide descriptive context. Martins and Rangel [3] noted that this structure significantly improves query efficiency in medical research by reducing complexity and providing a consistent analytical layer. In this study, the star schema consists of one fact table and seven dimension tables covering patient, doctor, hospital, condition, date, insurance, and admission data.

E. Data Visualization and Dashboard

Data visualization translates processed data into graphical representations that make patterns and trends accessible to non-technical stakeholders. Vornhagen et al. [8] identified through a scoping review that effective healthcare dashboards share four common design pillars: user involvement, actionable metrics, usability, and workflow integration. In this study, a single-page Power BI dashboard is built on these principles to present key healthcare analytics including patient risk distribution, billing patterns, and encounter trends, supported by interactive filters.

III. METHODOLOGY

A. Data Source and Data Generation

The dataset used in this study is the Healthcare Dataset published on Kaggle by prasad22, a synthetic dataset designed to simulate real-world hospital encounter records. It consists of 55,500 rows and 15 columns spanning clinical, demographic, and financial attributes of patient visits. Because the data is synthetically generated, it does not represent any real patient population but reflects the structure and variety typically found in hospital administrative systems, making it suitable for validating an end-to-end data engineering pipeline. The dataset

is relevant to this study because it covers both clinical dimensions such as medical condition, medication, and test results, and operational dimensions such as billing amount, admission type, and insurance provider, providing sufficient complexity to demonstrate ETL processing, dimensional modeling, and dashboard visualization.

TABLE I. DATASET COLUMN DESCRIPTION

Column Name	Data Type	Description
Name	String	Patient full name
Age	Integer	Patient age in years
Gender	String	Male or Female
Blood Type	String	A+, A-, B+, B-, O+, O-, AB+, AB-
Medical Condition	String	Primary diagnosis
Date of Admission	Date	Date patient was admitted
Doctor	String	Attending doctor name
Hospital	String	Hospital name
Insurance Provider	String	Insurance company name
Billing Amount	Decimal	Total billed amount in USD
Room Number	Integer	Assigned room number
Admission Type	String	Emergency / Urgent / Elective
Discharge Date	Date	Date patient was discharged
Medication	String	Prescribed medication
Test Results	String	Normal / Abnormal / Inconclusive

B. Data Ingestion

Data ingestion is implemented in Transformation_validation.ktr using Pentaho Data Integration through a full load strategy, where the complete source CSV file is read and processed on every pipeline execution. Figure 1 shows the step-by-step ingestion process, where a CSV File Input step reads all 55,500 records from healthcare_dataset.csv, parsing all 15 columns as String type initially to preserve raw values before type casting. The extracted records are passed through a Select Values step that renames column headers to standardized snake_case names for downstream processing, followed by Sort Rows and Unique Rows steps that eliminate duplicate entries based on a composite deduplication key of patient name, age, and gender, reducing the dataset from 55,500 to 50,000 unique records. The full load strategy was chosen because the dataset is static and the pipeline is designed to fully refresh all warehouse tables on every execution, ensuring complete consistency between the source data and the analytical layer.



Fig.1. Data Ingestion Flow

C. Data Storage

Data storage in this study is organized across a structured local folder hierarchy and two distinct layers in a MySQL database, each serving a different purpose in the pipeline. The local file system root. The rawData folder stores the original unmodified source file healthcare_dataset.csv. The processedData folder stores the transformed output file transformed_data.csv produced after cleaning and feature engineering. The starSchema folder stores flat-file backups of all seven dimension tables and the fact table, written in parallel with the MySQL load. The logs folder stores execution log files generated by healthcareJob.kjb for each pipeline run, providing an audit trail for monitoring and debugging.

Within MySQL, the first layer is a staging table named staging_healthcare, which stores all 43,659 validated records together with 17 derived columns produced during transformation. This staging layer acts as the single enriched source for all downstream dimension and fact table population, allowing the pipeline to be re-executed and debugged at the transformation level without affecting the analytical layer. The second layer is the analytical schema healthcare_dw, containing seven dimension tables and one fact table structured as a star schema and optimized for the multi-dimensional queries issued by the Power BI dashboard. This separation between raw staging output and the analytical schema ensures that the warehouse tables are only refreshed after a complete and successful transformation run.

TABLE II. LOCAL FILE SYSTEM STRUCTURE

Folder / File	Purpose
rawData\	Original source data, untouched
processedData\	Cleaned and enriched staging output
starSchema\	Flat-file backup of warehouse tables
logs\	Execution audit trail per run
Transformation_validation.ktr	ETL validation and transformation logic
dimensions.ktr	Dimension table building logic
factTable.ktr	Fact table building logic
healthcareJob.kjb	Job orchestration and automation
healthcareDashboard	Power BI dashboard file

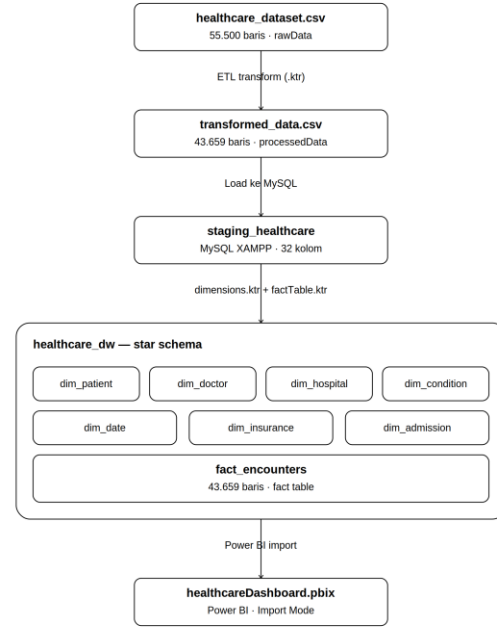


Fig.2. Data Ingestion Flow

D. Data Transformation

Transformation is implemented entirely within Transformation_validation.ktr and follows the sequential flow shown in Figure 3 and Figure 4. The process begins with two string normalization steps, Replace in string and String operations, which correct whitespace and formatting inconsistencies across all categorical fields. The cleaned stream then passes through four chained Value Mapper steps in sequence: gender valid, blood valid, admission valid, and test valid, each validating one categorical field against its allowed set of values and writing the result to a corresponding validity flag column. After all four validity flags are produced, Filter Invalid evaluates them together and splits the stream into two paths: records that fail any of the four checks are routed to the invalid output file, while records that pass continue to Select values 2 and Select values 3, which cast numeric and date fields to their correct data types. The stream then reaches Filter rows 2, which removes records failing null or range checks, routing the rejected records to invalid2.

Continuing from Filter rows 2, the surviving records pass through Dummy (do nothing), which serves as a junction point with no transformation, followed by Calculator and a first If field value is null step that fills missing values before the main risk and feature calculations. A second If field value is null 2 step is followed by Formula, which derives the initial set of formula-based columns. The output of Formula then passes through Admission Risk Score and Test Risk Score, two Value Mapper steps that convert admission_type and test_results into numeric risk components, followed by Calculator 2, which computes the remaining date-based derived fields. Before reaching the final formula stage, the stream passes through Filter rows 3, which performs a last validation check and routes any failing records to invalid4, while a separate branch from the

earlier null-handling stage routes its own rejected records to invalid3. The validated records then reach Formula 2, which calculates the final risk_score column, and the resulting stream is written simultaneously to two parallel outputs: Table output, which loads the data into the staging_healthcare MySQL table, and Text file output, which writes the same records to transformed_data.csv.

This structure shows that data rejection in the pipeline occurs at four distinct checkpoints rather than one, corresponding to the four invalid, invalid2, invalid3, and invalid4 output files visible in the canvas. Each checkpoint targets a different category of data quality issue: invalid captures categorical validation failures from the four chained Value Mapper checks, invalid2 captures null and range violations from Filter rows 2, invalid3 captures records failing the null-handling stage before the risk calculations, and invalid4 captures records failing the final Filter rows 3 check after risk scores have been computed. This four-point rejection design provides a more granular audit trail than a single consolidated filter would, since each rejected file corresponds to a specific stage of the transformation logic and makes it possible to trace exactly where in the pipeline a given record was excluded.

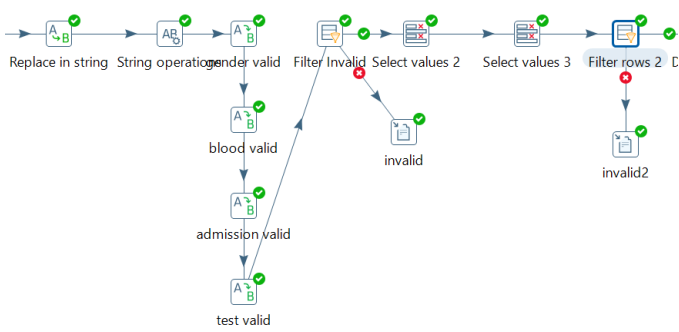


Fig.3. Data Transformation flow I

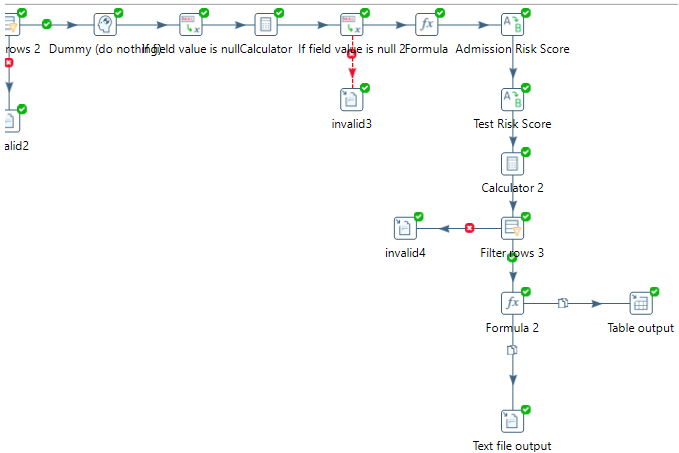


Fig.4. Data Transformation flow II

E. Data Modeling

The analytical data model follows the star schema pattern. Dimension tables are built by dimensions.ktr, which begins with a single Table Input reading from staging_healthcare and

fans out into seven parallel Select Values branches, each extracting and renaming columns for its respective dimension. Before any dimension is loaded, an Execute SQL script step inside dimensions.ktr disables foreign key checking and truncates the fact table using SET FOREIGN_KEY_CHECKS = 0, TRUNCATE TABLE fact_encounters, and SET FOREIGN_KEY_CHECKS = 1. Each branch then passes through a Unique Rows HashSet step and an Add Sequence step to assign surrogate keys, before writing output to both a CSV file and a MySQL dimension table in parallel.

The dim_date branch additionally applies a Calculator step to derive day_of_year and week_of_year, a Formula step to derive quarter and weekday_name, and a Value Mapper step to derive month_label from admit_month. A final Select Values step orders the ten resulting columns as date_id, admit_year, quarter, admit_month, month_label, week_of_year, weekday_name, weekday_num, day_of_year, and full_date. The fact table fact_encounters contains only foreign keys and six measurable attributes. Foreign key values are resolved in factTable.ktr through seven Database Join steps, each querying the corresponding MySQL dimension table with a parameterized WHERE clause.

After all seven Database Join steps complete, a Select Values step retains only the 13 columns that belong in the fact table: patient_id, doctor_id, hospital_id, condition_id, date_id, insurance_id, admission_id, billing_amount, length_of_stay_days, admission_risk_score, test_risk_score, risk_score, and conversion_rate_final. An Add Sequence step then generates fact_id as an auto-incrementing integer starting from 1.

TABLE III. DIMENSION BRANCH COLUMN MAPPING

Dimension	Columns Staging from	Renamed To	Surrogate Key
dim_patient	patient_name, age, gender, blood_type, age_group	-	patient_id
dim_doctor	doctor	-	doctor_id
dim_hospital	hospital, room_number	-	hospital_id
dim_condition	medical_condition, medication, test_results, condition_category	-	condition_id
dim_date	date_of_admission, admission_year, admission_month, day_of_week_num, date_key	full_date, admit_year, admit_month, weekday_num, date_id	date_id
dim_insurance	insurance_provider	-	insurance_id
dim_admission	admission_type	-	admission_id

TABLE IV. DATABASE JOIN QUERIES IN FACTTABLE.KTR

DB Join Step	SQL Query	Parameters
joinPatient_id	SELECT patient_id FROM dim_patient WHERE patient_name=? AND age=? AND gender=? LIMIT 1	patient_name, age, gender

doctor_id	SELECT doctor_id FROM dim_doctor WHERE doctor=? LIMIT 1	doctor
hospital_id	SELECT hospital_id FROM dim_hospital WHERE hospital=? AND room_number=? LIMIT 1	hospital, room_number
condition_id	SELECT condition_id FROM dim_condition WHERE medical_condition=? AND medication=? AND test_results=? LIMIT 1	medical_condition, medication, test_results
date_id	SELECT date_id FROM dim_date WHERE date_id=? LIMIT 1	full_date, admit_year, admit_month, weekday_num, date_id
insurance_id	SELECT insurance_id FROM dim_insurance WHERE insurance_provider=? LIMIT 1	insurance_provider
admission_id	SELECT admission_id FROM dim_admission WHERE admission_type=? LIMIT 1	admission_type

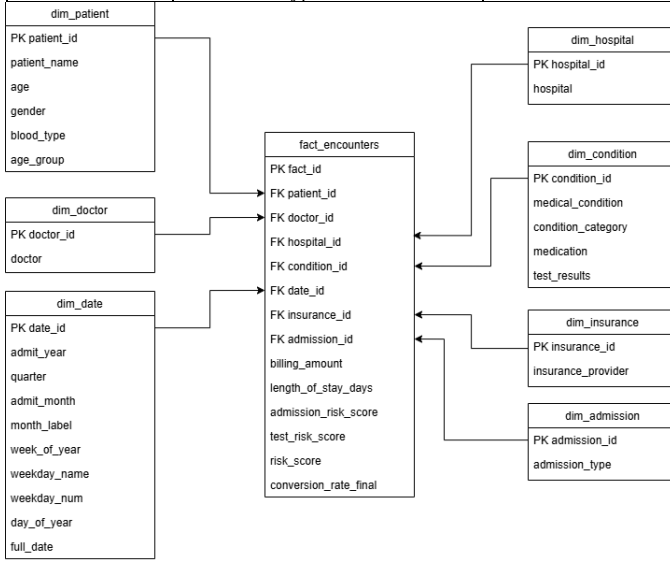


Fig.5. Star Schema Diagram

TABLE V. STAR SCHEMA SUMMARY

Table	Rows	Primary Key	Columns
dim_patient	43,542	patient_id	6
dim_doctor	43,601	doctor_id	2
dim_hospital	43,601	hospital_id	3
dim_condition	3,654	condition_id	5
dim_date	1,827	date_id	10
dim_insurance	5	insurance_id	2
dim_admission	3	admission_id	2
fact_encounters	43,659	fact_id	14

F. Data Quality and Validation

Data quality is enforced across three distinct layers of the pipeline rather than as a single isolated check. At the ingestion

layer, Sort Rows and Unique Rows eliminate 5,500 duplicate records using a composite key of patient_name, age, and gender. At the transformation layer, four Value Mapper steps enforce allowed-value constraints on gender, blood type, admission type, and test results; a Filter Rows step enforces null checks on eight critical fields and range constraints on age and billing_amount; and a second Filter Rows step eliminates records with negative length_of_stay_days. Records rejected at each stage are written to dedicated audit output files rather than silently dropped, providing a complete traceability trail. At the loading layer, MySQL foreign key constraints on all seven FK columns in fact_encounters enforce referential integrity, ensuring that no fact row can reference a dimension entry that does not exist.

TABLE VI. DATA QUALITY VALIDATION RULE

Layer	Field	Rule	Action if Fail
Ingestion	patient_name, age, gender	No duplicate combination	Remove row
Transformation	gender_valid	Must not be invalid	Route to invalid.csv
Transformation	blood_type_valid	Must not be invalid	Route to invalid.csv
Transformation	admission_type_valid	Must not be invalid	Route to invalid.csv
Transformation	test_result_valid	Must not be invalid	Route to invalid.csv
Transformation	8 required fields	IS NOT NULL	Route to invalid2.csv
Transformation	age	Within valid range	Route to invalid2.csv
Transformation	billing_amount	> 0	Route to invalid2.csv
Transformation	length_of_stay_days	>= 0	Remove row
Loading	All FK columns	Must reference existing row	Constraint violation

TABLE VII. DATA QUALITY PIPELINE SUMMARY

Stage	Input Rows	Output Rows	Removed	Reason
CSV Ingestion	55,500	55,500	0	Initial load
Deduplication	55,500	50,000	5,500	Duplicate composite key
Categorical Validation	50,000	43,741	6,259	Invalid category values
Null and Range Validation	43,741	43,659	82	Null or out-of-range fields
Final Staging Output	43,659	43,659	0	Loaded to staging_healthcare

G. Automation and Orchestration

Pipeline automation is implemented in healthcareJob.kjb, which sequences execution of three transformation entries using On Success and On Failure hops. The job starts at the Start entry and proceeds unconditionally to Transformation_validation. On success, it continues to dimensions; on failure, it routes to the failedTransformation Write to Log entry with subject 'FAILED]

Transformation_validation.ktr and log level Error, after which the job stops. The same pattern applies for dimensions and factTable. The SQL truncation that clears the fact table before dimension loading is embedded inside dimensions.ktr itself as an Execute SQL script step, so it executes consistently regardless of whether dimensions.ktr is invoked from the job or run independently from Spoon. This design ensures that both execution paths, automated orchestration and manual execution, produce a consistent and clean refresh of all warehouse tables.

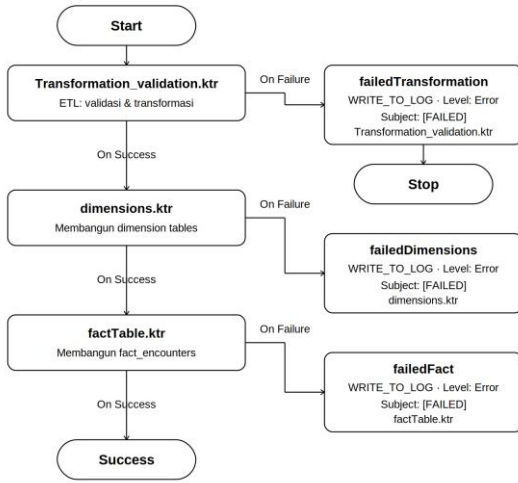


Fig.6. Job Orchestration Flowchart

TABLE VIII. JOB ENTRY CONFIGURATION

Entry Name	Type	Triggered By	On Success	On Failure
Start	SPECIAL	Manual execution	Transformation_validation	-
Transformation_validation	TRANS	Start (unconditional)	dimensions	failedTransformation
dimensions	TRANS	Validation on success	factTable	failedDimensions
factTable	TRANS	Dimensions success	Success	failedFact
Success	SUCCESS	factTable success	-	-
failedTransformation	WRITE_TO_LOG	Validation on failure	-	-
failedDimensions	WRITE_TO_LOG	Dimensions failure	-	-
failedFact	WRITE_TO_LOG	factTable failure	-	-

H. Data Consumption

The analytical data in healthcare_dw is served through Power BI Desktop connected directly to MySQL using Import mode. Seven Many-to-One relationships are created between

fact_encounters and each of the seven dimension tables, with single-direction cross-filtering. Six DAX measures and two calculated columns, Risk Category and Billing Band, are created to power the dashboard visualizations.

The dashboard occupies a single page and is organized into three functional rows beneath a title and a slicer row. The top row contains four interactive slicers bound to medical_condition from dim_condition, admit_year from dim_date, insurance_provider from dim_insurance, and hospital from dim_hospital, which filter all visuals on the page simultaneously. Below the slicers, four KPI cards display Total Encounters, Total Billing, Avg Length of Stay, and High Risk Encounters. The next row contains two visuals side by side: a donut chart titled Risk Patient Distribution that breaks down Total Encounters by the Risk Category calculated column, and a horizontal clustered bar chart titled Average Billing by Medical Condition that plots Avg Billing per Encounter against medical_condition from dim_condition. The final row also contains two visuals: a line chart titled Monthly Visit Trends that plots Total Encounters against month_label from dim_date with admit_year providing year-level breakdown, and a donut chart titled Patient Distribution by Age Group that breaks down Total Encounters by age_group from dim_patient.

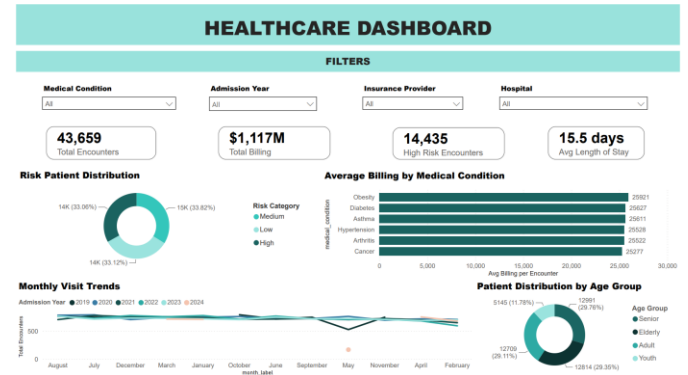


Fig.7. Dashboard Layout

IV. RESULT AND DISCUSSION

A. Pipeline Execution Result

The complete pipeline was executed through healthcareJob.kjb and successfully processed all three stages in sequence without manual intervention. The execution log confirms that Transformation_validation.ktr completed in approximately 15 seconds, reading 55,500 input rows and writing 43,659 validated rows to both transformed_data.csv and the staging_healthcare MySQL table. The dimensions.ktr transformation completed in under 2 seconds, populating all seven dimension tables from the single staging source through the fan-out branching design. The factTable.ktr transformation, which relies on seven sequential Database Join steps to resolve foreign keys against MySQL, took considerably longer at approximately 5 minutes and 30 seconds for the same 43,659 rows, since each Database Join issues one parameterized query per row against its respective dimension table.

In an iterative development, two classes of failure to execute were noted and addressed. The first was a column name mismatch error, Unknown column test_results in field list, which happened because the column name was test_result in the table dim_condition in MySQL, but Pentaho named it test_results. This was fixed by mapping the MySQL column name to the Pentaho field name. It was the second failure that proved more significant, Cannot delete or update a parent row, when trying to truncate a dimension table, while there were still rows in fact_encounters that referenced it. This was solved by adding a SET FOREIGN_KEY_CHECKS = 0 and TRUNCATE TABLE fact_encounters sequence to the dimensions.ktr, which is executed before any dimension table is refreshed, even if the transformation is triggered by the job, as the fact table is cleared each time.

TABLE IX. PIPELINE EXECUTION SUMMARY

Stage	Execution Time	Rows In	Rows Out	Result
Transformation_validation.ktr	15 sec	55,500	43,659	Success
dimensions.ktr	1-2 sec	43,659	7 tables populated	Success
factTable.ktr	5 min 30 sec	43,659	43,659	Success
Total healthcareJob.kjb run	6 min	55,500	43,659 final facts	Success

B. Data Quality Outcomes

Using the multi-stage validation strategy outlined in Chapter 3, the dataset size could be measured and documented at each step of the procedure. The number of patient records that were deduplicated due to matching patient name, age and gender accounted for 5,500 of the initial 55,500 records. The remaining 6,259 were removed during categorical validation for gender, blood type, admission type or test result fields with invalid values. 82 more records were removed by null and range checking. In total, 43,659 records, 78.7 percent of the original data, were kept and placed into the data warehouse.

This result illustrates the fact that without a structured and multi-layered validation strategy, over 20% of the raw data would be in an unusable state in the analytical layer, as either an exact duplicate or as data containing logically invalid categorical or numeric values. Another benefit of the approach taken, and an important one for development, was to route records to separate audit files, rather than simply discarding them; this meant that each record rejection could be examined and traced back to the validation rule used to reject it when debugging the pipeline during development.

TABLE IX. PIPELINE EXECUTION SUMMARY

Stage	Rows Remaining	Cumulative % of Original	Rows Removed at This Stage
Raw input	55,500	100.0%	-
After deduplication	50,000	90.1%	5,500

After categorical validation	43,741	78.8%	6,259
After null/range validation	43,659	78.7%	82

C. Fact Table Outcomes and Star Schema

The dimension-building phase yielded 7 dimension tables with cardinalities of less than 43,500 rows, which is due to the fact that patient identity [name, age, gender], as well as doctor name, are rather unique in the data. dim_date only has 1,827 rows, representing the number of distinct calendar dates that span the range of the data set, and the dim_insurance and dim_admission have only 5 and 3 rows, respectively, because they are low cardinality categorical attributes. This distribution will validate that the star schema design is correct, where the two tables that represent the entity dimensions hold large numbers of entities and the two tables representing the categorical dimensions hold relatively few entities that are valid dimensions.

The fact table fact_encounters was populated with the exact number of rows that were validated in the transformation stage, and all seven foreign key columns were resolved during the Database Join steps in factTable.ktr. There was no evidence that any foreign key values were null in the final fact table and it was determined that each record in the staging table was a match in each of the seven dimension tables. This result was confirmed by running a query on the fact table to see if there are rows with any foreign key column that is null, the result was 0 rows, and by running a full join across all eight tables, the result was the number of rows that should have been returned with all of the foreign keys resolved to the correct value.

D. Dashboard and Analytical Insights

The Power BI dashboard, directly connected to the healthcare_dw schema in Import mode, renders six DAX measures and two calculated columns, Risk Category and Billing Band, across five visualizations arranged on a single page. The four KPI cards at the top provide an at-a-glance operational summary, reporting 43,659 total encounters and a defined high-risk threshold at a risk_score of 1.2 or above through the High Risk Encounters measure. The Risk Patient Distribution donut chart breaks down Total Encounters by the Risk Category calculated column, which classifies each encounter as Low, Medium, or High based on boundaries derived from the actual distribution of risk_score in the fact table, ranging from 0.000 to 1.808 with a mean of 0.900, giving the segmentation a data-driven basis rather than an arbitrary one.

The Average Billing by Medical Condition bar chart plots the Avg Billing per Encounter measure against medical_condition from dim_condition, allowing direct comparison of treatment cost across diagnoses and supporting cost-management discussions for hospital administrators. The Monthly Visit Trends line chart plots Total Encounters against month_label from dim_date, with admit_year providing an additional year-level dimension, revealing the temporal distribution of patient volume across the dataset. The Patient Distribution by Age Group donut chart breaks down Total

Encounters by age_group from dim_patient, completing the demographic view of the patient population. The four slicers at the top of the page, bound to medical_condition, admit_year, insurance_provider, and hospital, filter all five visuals simultaneously, confirming that the seven Many-to-One relationships established in the data model correctly propagate filter context from each dimension table to the fact table and onward to every visual bound to a fact table measure.

V. CONCLUSION

A. Summary of Findings

This study set out to design and implement a complete, automated healthcare data pipeline integrating ETL processing, dimensional data modeling, and interactive dashboard visualization within a single coherent system, an objective achieved through the implementation described in this report. The ETL stage, built in Pentaho Data Integration, transformed 55,500 raw synthetic patient encounter records into 43,659 validated records through deduplication, categorical validation, type casting, and feature engineering. This validated data was then organized into a star schema comprising seven dimension tables and one measures-only fact table, with referential integrity enforced through MySQL foreign key constraints. A Pentaho Job orchestrated all three transformations sequentially, with structured error handling and logging at each stage, while the resulting data was presented through a single-page Power BI dashboard built on six DAX measures and two calculated columns, Risk Category and Billing Band, delivering five analytically prioritized visualizations supported by four interactive slicers.

The results confirm that the integrated pipeline architecture introduced in Chapter 1 was achieved as intended. The pipeline reliably transforms raw healthcare data into an analytically optimized warehouse, with the star schema supporting efficient multi-dimensional querying across all eight tables. The orchestration layer executes the pipeline end-to-end without manual intervention, correctly isolating and logging failures, while the dashboard delivers clinically and operationally relevant insights, including patient risk distribution, billing patterns by medical condition, and encounter trends over time.

B. Evaluation and Reflection

1) Tools and Technologies Chosen

For this implementation, it was decided to use three main technologies, each for a particular reason related to the constraints and goals of the project. The ETL layer was chosen as Pentaho Data Integration offers a visual, graph-based transformation design environment without the need to write custom extraction or transformation code and offers a complete set of ETL primitives for production-quality data cleaning, such as value mapping, calculator and formula steps, multi-path filtering, and parameterized database operations. MySQL was chosen for the storage layer because it is open source, well documented, and supports the Foreign Key Constraint Enforcement required by the star schema design for referential integrity. Microsoft Power BI was chosen for the consumption layer due to its ability to connect natively with MySQL, support for DAX to create custom measures and its interactive

dashboard experience with built in filtering capabilities, without the need to write a custom front end application.

2) Advantages of Design Decisions

This implementation had a number of elements that clearly benefited its design. Staging_healthcare served as the single source, instead of reading the source CSV over and over again, allowing for every dimension to be built from a single, consistent, and already validated set of data, preventing dimensions from drifting out of sync with one another. It was a good idea to write all the rejected records to an audit CSV file instead of silently discarding all the invalid records as it helped in development when I wanted to see which specific validation rule caused which record to be rejected. Since the logic for truncating the fact table is embedded in the dimensions.ktr rather than just being a separate step in the job, the transformation is more robust in that it truncates the fact table when dimensions.ktr is run as part of the orchestrated job, or even when it is run stand-alone for testing.

3) Trade-offs

The main drawback of this design is the choice of foreign key resolution strategy for factTable.ktr. Database Join was chosen over the other approach (Stream Lookup) because it involves fewer transformations on the transformation canvas and does not necessitate loading large dimension tables into memory prior to starting the processing. This convenience, however, came at a measurable performance cost: the factTable.ktr transformation took around five and a half minutes to process 43,659 rows of data, whereas it should only take less than a minute if Stream Lookup or a single SQL join had been used in place of the Table Input query. This was a compromise in this implementation as the data set is relatively small and the pipeline does not execute on a real-time schedule, but is a definite limitation that would need to be overcome if the same design were to be applied to a much larger data set or to a more frequent refresh cycle.

4) Limitations

The main disadvantage is that the entire load refresh approach is implemented throughout the pipeline. Currently, they are not able to support incremental updates or Slowly Changing Dimension tracking, which would be required if patient and encounter data were being continually updated in a realistic, production environment; all dimensions and the fact table are dropped and reloaded every time, so they remain consistent. The third constraint is that of the data set itself, namely how the data are organised in the resulting analytical output of the Dashboard is correct, but does not reflect real clinical patterns, and so should not be interpreted as a result of the actual clinical outcome.

This study has several additional constraints that place limits on the conclusions that can be drawn. Further, the data source is not a real-time or continuously changing data source, but a static synthetic data source, and the pipeline was tested with the full load batch flow only and not tested for streaming and incremental ingestion flows. Second, pipeline correctness and execution results is not formally evaluated in the study;

however, the study does not contain formal end user evaluation of the pipeline with actual clinical and/or administrative end user, and therefore, any claims about the practical usability of the pipeline are based on design principles found in the literature. Third, all pipelines were developed and executed locally on one machine using XAMPP and no testing was done under the concurrent multi-user load and distributed deployment condition, so the conclusions of the performance do not have any implications in the production scale.

ACKNOWLEDGMENT

The author thanks the academic advisor and the Information Technology Program at Bina Nusantara University for their guidance and support throughout the development of this research. The author also thanks to the author of the Healthcare Dataset published on Kaggle, whose synthetic patient encounter data made the implementation and evaluation of this pipeline possible. Appreciation is further extended to the open-source communities behind Pentaho Data Integration and MySQL, as well as to Microsoft Power BI, for providing the tools that enabled the design, transformation, and visualization stages of this study. The author also acknowledges the use of an AI-based assistant to support the drafting and editing process of this report.

REFERENCES

- [1] S. Imran, T. Mahmood, A. Morshed, and T. Sellis, "Big data analytics in healthcare: a systematic literature review and roadmap for practical implementation," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 1, pp. 1–22, Jan. 2021.
- [2] N. Cozzoli, F. P. Salvatore, N. Faccilongo, and M. Milone, "How can big data analytics be used for healthcare organization management? Literary framework and future research from a systematic review," *BMC Health Serv. Res.*, vol. 22, no. 1, p. 809, Jun. 2022.
- [3] T. G. S. Martins and F. S. Rangel, "Data warehouse and medical research," *Einstein (Sao Paulo)*, vol. 20, p. eED6324, Feb. 2022.
- [4] S.-T. Liaw, J. G. N. Guo, S. Ansari, J. Jonnagaddala, M. A. Godinho, A. J. Borelli, S. de Lusignan, D. Capurro, H. Liyanage, N. Bhattal, V. Bennett, J. Chan, and M. G. Kahn, "Quality assessment of real-world data repositories across the data life cycle: a literature review," *J. Am. Med. Inform. Assoc.*, vol. 28, no. 7, pp. 1591–1599, Jul. 2021.
- [5] A. Q. Hungbo, C. Adeyemi, and O. O. Ajayi, "Power BI-based clinical decision support system for evidence-based nurse decision-making," *Int. J. Adv. Multidiscip. Res. Stud.*, vol. 4, no. 6, pp. 2653–2668, 2024.
- [6] D. Seenivasan, "ETL (extract, transform, load) best practices," *Int. J. Comput. Trends Technol.*, vol. 71, no. 1, pp. 40–44, Jan. 2023, doi: 10.14445/22312803/IJCTT-V71I1P106.
- [7] R. Syed, R. Eden, T. Makasi, I. Chukwudi, A. Mamudu, M. Kamalpour, D. K. Geeganage, S. Sadeghianasl, S. J. J. Leemans, K. Goel, R. Andrews, M. T. Wynn, A. ter Hofstede, and T. Myers, "Digital health data quality issues: systematic review," *J. Med. Internet Res.*, vol. 25, p. e42615, Mar. 2023, doi: 10.2196/42615.
- [8] H. Vornhagen, S. Barrett, C. Carroll, L. K. Iladiva, G. Martin, D. McKeown, and J. Martin, "Design practices for data dashboards in health care: scoping review," *J. Med. Internet Res.*, vol. 28, p. e77361, Feb. 2026, doi: 10.2196/77361.